

The Growing Importance of Energy-Aware Numerical Precision in Scientific Computing

Eduards Chipatecua *, Juan Galvis *, and Francisco Gómez*

Abstract The rapid expansion of scientific computing, artificial intelligence, and large-scale data processing has transformed the landscape of computational efficiency. As computational resources become more powerful, their energy footprint grows proportionally, demanding a new paradigm in which energy efficiency is considered alongside numerical accuracy and scalability. In this work, we analyze the relationship between numerical precision and energy consumption in fundamental arithmetic operations, with a focus on CPU-based measurements using the Running Average Power Limit (RAPL) interface available on modern Intel and AMD processors. Three numerical experiments were conducted: (i) basic arithmetic operations on integer and floating-point types; (ii) prime number generation comparing exact (integer) and approximate (floating-point) workloads; and (iii) LU factorization with single, double, and mixed precision, including iterative refinement. The results show that floating-point arithmetic consistently consumes more energy than integer arithmetic, with the exception of division. This behavior is evident in our basic arithmetic experiments and especially in prime number generation, where the use of floating-point computations proved more efficient in both runtime and energy consumption. Meanwhile, increasing the word length from 32 to 64 bits has only a marginal impact. In the LU tests, single precision achieved the best trade-off between accuracy and energy efficiency, whereas mixed-precision refinement on CPU introduced additional overhead without improving performance. These findings underscore the need to integrate energy consumption into the evaluation of numerical algorithms. Considering energy efficiency as a central criterion is essential for sustainable high-performance computing and motivates a reformulation of classical computational complexity theory to include energy as a primary metric.

AMS Subject Classification: 65Y20, 65G50, 68M15.

Key words: Energy-aware computing, Numerical precision, Floating-point arithmetic, Scientific computing, Sustainability.

Departamento de Matemáticas, Universidad Nacional de Colombia, Carrera 45 No. 26-85, Edificio Yu Takeuchi, Bogotá D.C., Colombia, {emendezc, jcgaltisa, fagomezj}@unal.edu.co.

1 Introduction

Mathematical modeling lies at the heart of scientific computation. The standard modeling pipeline—from the real-world problem to the computed solution—involves successive transformations: formulation of the mathematical model, abstraction, numerical approximation, algorithmic design, and finally, implementation on a classic computing device. Each stage introduces specific sources of error: modeling errors from simplifications, discretization errors from numerical schemes, and rounding or truncation errors from the finite representation of numbers in computers [9, 25, 8].

The accelerated growth of scientific computing, artificial intelligence, and cloud services has profoundly transformed both the scale and nature of digital processing. Today, each advance in computational capacity carries an energy cost that can no longer be considered marginal: global data centers consume energy on the scale of small cities [3], and the carbon footprint associated with the training of large machine learning models has exposed an unavoidable reality, the computational efficiency should also involve the energy consumption, [18].

Half of this modeling process today operates within a classic computing context. The final step, where algorithms are executed, relies fundamentally on the architecture of modern hardware—central processing units (CPUs) and graphics processing units (GPUs)—and the finite-precision arithmetic these systems employ. Computers represent real numbers using floating-point formats that follow standards such as IEEE 754 [12]. Numerical data in digital computers are represented either as integers or as floating-point numbers. An n -bit signed integer is encoded in two's complement form as

$$x = -s 2^{n-1} + \sum_{i=0}^{n-2} a_i 2^i,$$

where s is the sign bit and $a_i \in \{0, 1\}$. Floating-point numbers, on the other hand, follow the IEEE 754 standard [12] and are expressed as

$$x = (-1)^s (1 + m) 2^{e-\Delta},$$

where m is the fractional part of the mantissa, e the exponent, and Δ a bias term. The most common formats include the classical `float16`, `float32`, and `float64`, as well as modern variants such as `bfloat16` (Brain Floating Point) [27] and NVIDIA's `TensorFloat-32` (TF32) [13]. See Table 1. Each format represents a compromise between numerical precision, computational speed, and energy efficiency. These representations determine not only numerical range and precision but also the computational and energetic cost of arithmetic operations.

While `float64` (double precision) has long been the standard in classical scientific computing applications such as fluid flow modeling, structural mechanics, and computational design, recent trends in large-scale machine learning and optimization favor reduced-precision arithmetic (e.g., `float32`, `bfloat16`, or `float16`); [5, 16]. In these applications, performance gains are achieved through hardware acceleration and reduced memory bandwidth without significant degradation of

Table 1 Most common numerical data types and their bit allocation, decimal precision, and approximate dynamic range. Floating-point formats follow the IEEE 754 standard [12], with modern variants such as bfloat16 [27] and TensorFloat-32 (TF32) [13] introduced for efficient computation on contemporary accelerators.

Type	Bits	Sign	Mantissa	Exp	Decimals	Approx. Range
Int32	32	1	—	—	9–10 digits	$[-2^{31}, 2^{31} - 1]$
Int64	64	1	—	—	18–19 digits	$[-2^{63}, 2^{63} - 1]$
float16 (IEEE)	16	1	10	5	≈ 3.3	10^{-5} to 10^5
bfloat16	16	1	7	8	≈ 2.4	10^{-38} to 10^{38}
float32	32	1	23	8	≈ 7.2	10^{-38} to 10^{38}
float64	64	1	52	11	≈ 15.9	10^{-308} to 10^{308}
TF32 (NVIDIA)	19	1	10	8	≈ 3	10^{-38} to 10^{38}
FP8-E4M3 (NVIDIA)	8	1	3	4	≈ 1	10^{-5} to 10^2
FP8-E5M2 (NVIDIA)	8	1	2	5	≈ 0.6	10^{-6} to 10^4

convergence properties or predictive accuracy [17]. Nevertheless we think that the community lacks a unified consensus regarding when and how to employ mixed or reduced-precision arithmetic in numerical simulations. The appropriate choice depends strongly on problem conditioning, stability properties of the algorithm, and sensitivity to rounding errors.

Systematically reporting the energy consumption associated with numerical operations is not merely a methodological improvement but a necessity for advancing toward truly sustainable computing. Incorporating this type of analysis will potentially enable the design of algorithms, libraries, and architectures that optimize not only performance but also energy cost per operation. In particular, exploring how numerical precision influences energy consumption provides a concrete and measurable starting point for understanding the relationships among accuracy, efficiency, and computational sustainability.

2 Related Work

The relationship between energy consumption and programming languages has been studied previously. Influential work, such as the research line in [19], evaluated the energy consumption of different languages, comparing up to twenty-seven of them. These studies established rankings of languages by energy efficiency, execution time, and memory usage. Complementing this perspective, [26] presents a framework for measuring energy in compiled programs and shows clear differences between languages even when implementing equivalent algorithms. These results highlight that the choice of programming language can have a direct impact on energy consumption, motivating more targeted analyses. In parallel, other studies have explored how numerical precision affects both performance and energy consumption in high-performance applications. The study presented in [20] evaluates the impact of reducing precision when solving the Poisson equation using ICCG

and when performing ground motion simulations with the Adaptive CG solver [21], comparing double precision (64-bit) and single precision (32-bit) arithmetic. The findings confirm that lower precision can significantly shorten execution times and substantially decrease energy consumption.

Our work extends these research directions in two ways. First, we analyze the effect of numerical precision on energy consumption specifically in Julia, a modern scientific computing language [4, 10, 24] that was not included in prior studies. Second, we broaden the scope of the analysis by evaluating not only single and double floating-point precision, but also various integer formats, across several experimental scenarios that enable a more detailed characterization of how numerical formats influence energy consumption. These scenarios include basic arithmetic operations on integer and floating-point types, prime number generation comparing exact integer-based workloads with approximate floating-point ones, and the solution of linear systems via LU factorization in single, double, and mixed precision. This collection of experiments provides a more granular view of how different numerical formats interact with language and hardware characteristics to determine energy consumption.

3 Methodology and Experimental Design

This section details the experimental procedure used to quantify energy consumption in our study. All executions were carried out under strictly controlled conditions to reduce variability associated with thermal factors and system scheduling. Both the energy consumed and the execution times were systematically recorded and subsequently averaged over multiple repeated trials, which ensured the statistical validity of the results. Although the measurements were not directly cross-checked with specialized equipment, the literature indicates that this approach provides estimates that are close to and sufficiently representative [7, 19, 26, 20].

Energy consumption was measured by directly accessing the counter exposed by Intel RAPL [14] in the Linux file system, located at:

```
/sys/class/powercap/intel-rapl:0/energy_uj
```

Within the RAPL hierarchy, the core domain represents the energy consumed exclusively by the processing cores of the CPU package. This domain encompasses the execution units (ALU and FPU), the L1 and L2 caches of each core, as well as the internal buffers and associated control logic. In our experiments, the value read comes directly from the counter corresponding to this core domain, which accumulates energy consumption since system boot and reports it in microjoules (J).

For each program execution under test, the counter value was recorded immediately before and after execution, and the difference between the two readings was computed. This difference corresponds to the total energy consumed during the execution interval, expressed in joules.

Analogously, the execution time was measured using timestamps, which returns the current time in seconds with nanosecond precision. A timestamp was recorded before starting the program and another after its completion; their difference represents the total execution duration.

Finally, each experiment was repeated 1,000 times sequentially, discarding the initial compiler startup time to avoid bias in the measurements.

Table 2 Specifications of the system used in the experiments.

Component	Specification
Operating System	Ubuntu 24.04.3 LTS (x86_64)
Kernel	Linux 6.14.0-33-generic
Processor (CPU)	AMD Ryzen 5 5600U with Radeon Graphics (6 cores, 12 threads)
Graphics (GPU)	AMD Radeon Vega Series / Radeon Graphics
RAM Memory	8 GB
Host	HP Pavilion Aero Laptop 13-be0

The experiments were performed on a laptop equipped with an AMD Ryzen 5 5600U processor featuring six physical cores and twelve threads, integrated Radeon Graphics, and 8 GB of RAM. The system operated under Ubuntu 24.04.3 LTS with kernel version 6.14.0-33-generic. The computing environment is summarized in Table 2. All routines were implemented in the Julia programming language (version 1.11.6), which internally uses the LLVM compiler [15] infrastructure to generate machine code using a *Just-In-Time* (JIT) compilation model.

4 Experimentos

This section describes the three experiments designed to evaluate the impact of numeric type and precision on execution time and energy consumption. Each experiment was fully implemented in Julia and executed under the controlled conditions described in Section 3. The experiments cover basic arithmetic operations, prime number generation, and the solution of linear systems through LU factorization at different precision levels.

The first experiment evaluates the time and energy cost of performing elementary arithmetic operations (addition, multiplication, and division) under different numeric formats: `Int32`, `Int64`, `Float32`, and `Float64`. A list of 19 values was constructed for each numeric type, and a doubly nested loop applied the corresponding operation to every pair of elements. This produces exactly $19 \times 19 = 361$ operations per iteration. The entire block is repeated one million times, yielding 361 million operations per program run. This procedure was applied independently to each combination of operation and numeric type. The design enables isolating differences between the processor’s arithmetic units (the ALU for integer operations and the FPU for

floating-point operations) and examining how their energy and time requirements vary with changes in precision or numeric representation.

The second experiment evaluates a classical prime-number generation algorithm. The algorithm proceeds incrementally: it begins with a list of known primes and, for each candidate number, checks whether it is divisible by any previous prime whose value does not exceed its square root. If no divisor is found, the candidate is added to the list. This method is highly division-intensive, which makes it an appropriate scenario for studying energy differences between integer and floating-point division. The algorithm was executed under four numeric configurations (`Int32`, `Int64`, `Float32`, and `Float64`), generating in each case the same fixed number of prime numbers to enable a consistent comparison across formats.

The third experiment analyzes the solution of linear systems via LU factorization under three configurations: single precision (`Float32`), double precision (`Float64`), and mixed precision with iterative refinement (`Float32 + Float64`). The procedure begins with the construction of an $n \times n$ Frank matrix. This matrix is multiplied by a vector of ones to generate a system whose exact solution is known. Then:

- The LU factorization is performed at the corresponding precision level.
- The system is solved through forward and backward substitution.

In the mixed-precision variant, the procedure follows the classical iterative-refinement scheme. The system factorization is kept in single precision (`Float32`), while at each iteration the residual is evaluated in double precision (`Float64`), the correction is computed using the single-precision factorization, and the solution is updated in double precision. Since LU factorization of a dense matrix requires $O(n^3)$ operations, this experiment involves millions of arithmetic operations and represents a computationally intensive scenario that enables the evaluation of significant energy-consumption differences across precision levels.

All experiments and related materials are available in the following repository: <https://github.com/eguar11011/NumPrecEnergy>.

5 Results

This section presents the experimental results of the experiments presented in Section 4, focusing on execution time and energy consumption across different numerical types and precisions. By comparing integer and floating-point arithmetic in 32-bit and 64-bit configurations, we assess how arithmetic representation influences computational performance and energy efficiency on modern CPUs.

5.1 Basic Arithmetic Operations (Addition, Multiplication, and Division) in Different Numerical Types

This subsection presents a detailed comparison of execution time and energy consumption for basic arithmetic operations—addition, multiplication, and division—performed under different numerical representations and precisions. The objective is to analyze how the choice between integer and floating-point arithmetic, as well as between 32-bit and 64-bit precision, influences both computational efficiency and energy usage. By contrasting these configurations, the analysis highlights fundamental trade-offs in processor utilization, memory bandwidth, and arithmetic unit performance that arise when varying numerical formats.

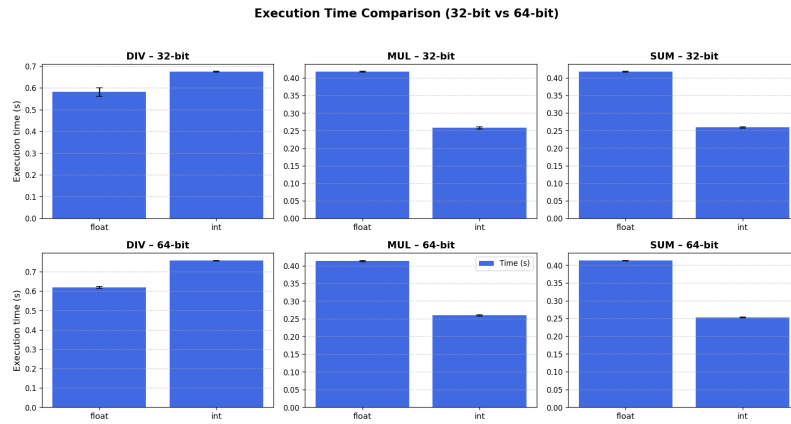


Fig. 1 Execution time comparison between integer and floating-point arithmetic for basic operations (division, multiplication, and addition) under 32-bit and 64-bit precision. Each bar represents the mean execution time over 10,000 runs, showing that integer operations are generally faster, except for division, where floating-point arithmetic performs better due to specialized FPU optimizations.

The analysis of energy consumption reveals a clear trend: operations performed using floating-point numbers demand more energy than those performed with integers. This difference is particularly noticeable in division operations, whereas in multiplication and addition the disparity is more moderate.

Similarly, increasing precision—from 32 to 64 bits—does not lead to a significant rise in energy consumption, although a slight upward trend can be observed. This behavior suggests that the larger word size introduces a marginal additional cost without substantially altering the overall energy profile.

Division consistently emerges as the most energy-intensive operation across all configurations. In 32-bit arithmetic, floating-point division requires roughly 20% more energy than integer division, and a similar disparity is observed at 64 bits, although both precisions show a modest increase relative to their 32-bit counterparts. For multiplication, the differences in energy consumption are less pronounced: in-

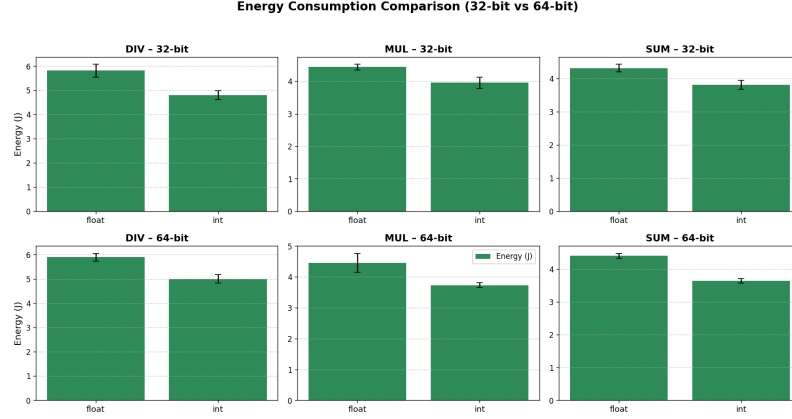


Fig. 2 Energy consumption comparison between integer and floating-point arithmetic for the same operations and precision levels as in Fig. 1. Floating-point operations consistently consume more energy than their integer counterparts, particularly in division, while the impact of increasing precision from 32-bit to 64-bit remains marginal.

teger arithmetic remains more efficient, and increasing the precision from 32 to 64 bits does not substantially alter this behavior. Addition exhibits the smallest gap between floating-point and integer arithmetic, yet retains a consistent advantage for integers, reinforcing their superior energy efficiency across basic workloads. In terms of execution performance, the overall trend reverses: integer operations are generally faster than floating-point ones, with division representing the principal exception. Indeed, floating-point division outperforms integer division in runtime due to specific optimizations within the floating-point unit (FPU), whereas integer division incurs additional overhead from rounding and remainder handling. Conversely, multiplication and addition are markedly faster when performed on integers, reflecting the greater computational simplicity of ALU-based arithmetic compared to floating-point operations, which involve mantissa normalization, exponent alignment, and rounding. (See Figures 3.)

5.1.1 Comparative Analysis Across All Operations

This section summarizes the global comparison of arithmetic performance and energy consumption across all benchmarked operations.

Figures 3 and 4 present the comparison of the **average energy consumption** obtained from different scripts implementing basic arithmetic operations—division, multiplication, and addition—using both single- and double-precision data types, in floating-point and integer representations. Complementarily, Figure 2 shows the **execution times** corresponding to the same experiments.

Regarding **energy consumption**, it is evident that *division* operations are the most demanding, reaching values in the range of **5.8 to 6.0 J** for floating-point

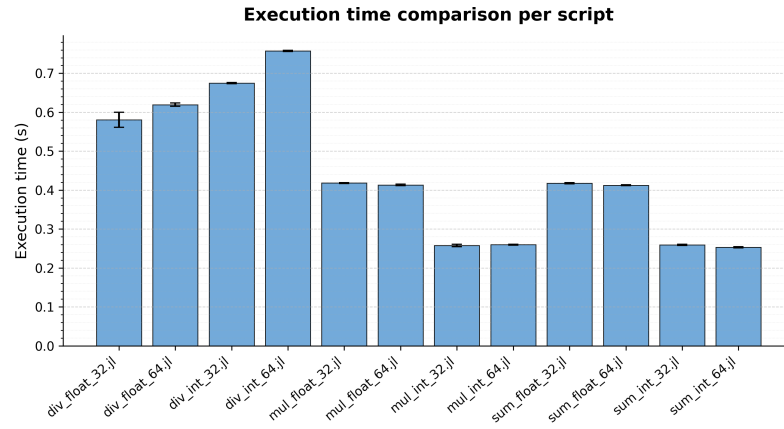


Fig. 3 Execution time comparison across all benchmark scripts for division, multiplication, and addition using integer and floating-point types in both 32-bit and 64-bit precision. Each bar represents the mean execution time measured over 10,000 iterations. Floating-point divisions execute faster than integer divisions, while integer operations maintain superior performance in multiplication and addition, demonstrating the architectural efficiency of ALU-based arithmetic.

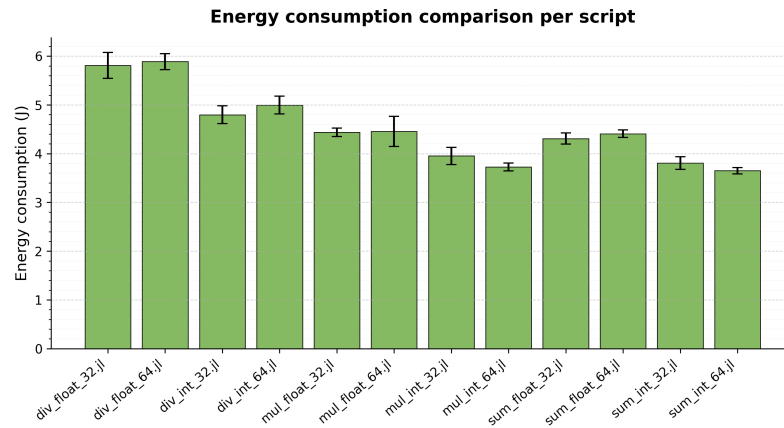


Fig. 4 Energy consumption comparison across all benchmark scripts for division, multiplication, and addition using integer and floating-point types in 32-bit and 64-bit precision. Each bar represents the mean energy consumption measured with Intel RAPL over 10,000 iterations. Floating-point operations exhibit consistently higher energy usage than integer ones, particularly in division, while integer arithmetic maintains lower and more stable energy profiles across all tested precisions.

cases (`div_flot_32.jl` and `div_flot_64.jl`). Integer divisions (`div_int_32.jl` and `div_int_64.jl`) also exhibit high energy consumption, although slightly lower (~4.8–5.0 J). In contrast, *multiplication* and *addition* operations are considerably more efficient. In particular, integer multiplications and additions (`mul_int_32.jl`, `mul_int_64.jl`, `sum_int_32.jl`, `sum_int_64.jl`) show the lowest consumption values, around **3.6–3.8 J**. Overall, the results confirm that **floating-point operations** entail higher energy costs than their **integer counterparts**, while the difference between **32-bit and 64-bit precisions** remains marginal and does not significantly affect total consumption.

In terms of **execution time**, a similar trend is observed, with a single exception: *floating-point divisions* execute faster than integer divisions. Nevertheless, divisions remain the most time-expensive operations, with durations between **0.6 and 0.75 s**, particularly in 64-bit integer division. Conversely, *multiplication* and *addition* operations exhibit substantially shorter times. The scripts operating with integers (`mul_int_*` and `sum_int_*`) demonstrate the best performance, with execution times ranging from **0.25 to 0.28 s**, while floating-point versions show slightly higher times, around **0.42 s**.

5.2 Prime Number Generation – Comparison Between Integer and Floating-Point Arithmetic (32-bit and 64-bit)

- **Integers (Int32 vs. Int64):**

No significant difference was observed in either execution time or energy consumption between the two integer sizes. This indicates that, for this particular algorithm, the computational cost of manipulating 32-bit and 64-bit integers is comparable on the evaluated architecture.

- **Floating-Point Numbers (Float32 vs. Float64):**

In contrast, clear differences were observed. Using **Float32** reduces both execution time and energy consumption compared to **Float64**. This suggests that lower-precision operations utilize the processor's floating-point unit (FPU) more efficiently, as they require less memory bandwidth and fewer compute cycles per operation.

5.3 LU Factorization in Single, Double, and Mixed Precision

- **Single Precision (32-bit):**

Exhibits the lowest execution time and energy consumption, although with reduced accuracy in the computed solution.

- **Double Precision (64-bit):**

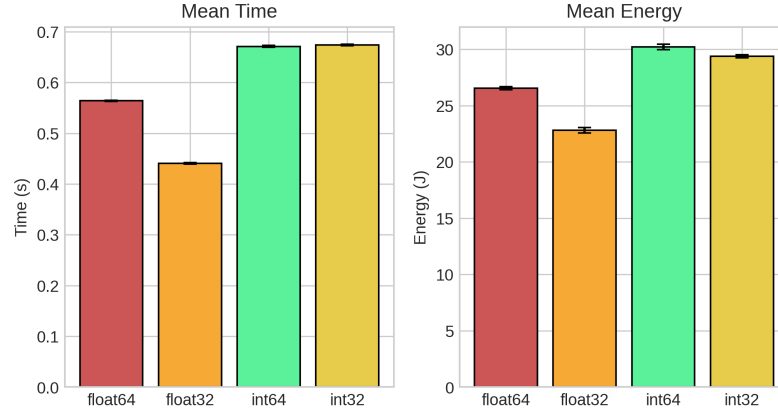


Fig. 5 Average execution time and energy consumption in prime number computation, calculated up to the 1,000,000th prime.

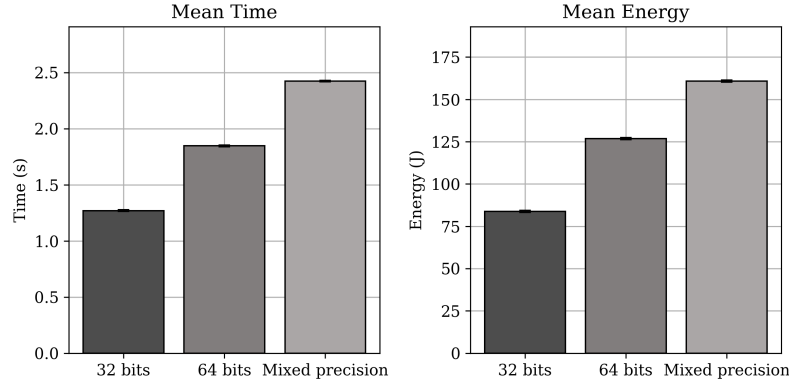


Fig. 6 Average execution time and energy consumption for LU factorization of a $10,000 \times 10,000$ matrix under single (32-bit), double (64-bit), and mixed precision configurations. The single-precision implementation exhibits the lowest computational cost, while the mixed-precision approach—despite its theoretical advantage—shows higher time and energy usage due to the additional overhead of iterative refinement on the CPU architecture.

Both execution time and energy consumption increase relative to single precision, but the resulting solution is more stable and accurate, as expected due to reduced rounding errors.

- **Mixed Precision:**

Contrary to expectations, on CPU the cost of mixed precision was found to be **higher in both time and energy** than in single and double precision. This suggests that, on this architecture, iterative refinement introduces an additional computational overhead that is not offset by the gains in numerical stability.

6 Conclusions

The experiments conducted in this study demonstrate that numerical precision and data representation exert a clear and measurable influence on both execution time and energy consumption. Integer arithmetic consistently exhibits lower energy usage and faster performance than floating-point arithmetic, while the impact of increasing precision from 32 to 64 bits remains marginal in most scenarios. In more complex tasks, such as LU factorization, single precision provides an attractive balance between efficiency and accuracy, whereas mixed-precision iterative refinement on CPU introduces additional overhead that can offset its theoretical advantages.

These findings highlight a broader implication: as scientific computing continues to scale, energy efficiency must become a central component of algorithmic analysis. Classical computational complexity, traditionally centered on time and memory, is no longer sufficient to capture the operational cost of modern numerical simulations. The results presented here support the emerging need for an extended complexity framework in numerical analysis—one that explicitly incorporates energy as a first-class metric. Advancing this perspective will be essential for designing sustainable algorithms and for understanding the trade-offs introduced by modern hardware and precision formats. Future work should expand these measurements to GPUs, heterogeneous systems, and large-scale distributed environments, where energy considerations may interact even more strongly with precision choices.

Acknowledgments

The authors thank the organizers of the Central America and the Caribbean Conference on Pure and Applied Mathematics (CMAM 2025) for the invitation to contribute to this volume and for providing a valuable forum for scientific exchange. Meetings like this are essential for strengthening collaboration and advancing mathematical sciences across Latin America and the Caribbean.

We also extend our special thanks to Juliana Nieto and César Pedraza for their valuable recommendations and suggestions on this paper, which significantly contributed to improving its content.

References

1. A. Abdelfattah, H. Anzt, E. G. Boman, E. Carson, *et al.*, “A Survey of Numerical Methods Utilizing Mixed-Precision Arithmetic,” *The International Journal of High Performance Computing Applications*, vol. 35, no. 4, pp. 344–369, 2021. doi: <https://doi.org/10.1177/10943420211003311>.
2. A. Fog, “Instruction Tables: Lists of Instruction Latencies, Throughputs and Micro-operation Breakdowns for Intel, AMD and VIA CPUs,” *Agner.org*, 2024. Available: https://agner.org/optimize/instruction_tables.pdf

3. X. Chen, X. Wang, A. Colacelli, M. Lee, and L. Xie, “Electricity Demand and Grid Impacts of AI Data Centers: Challenges and Prospects,” *arXiv preprint arXiv:2509.07218v3*, 2025. <https://arxiv.org/abs/2509.07218v3>.
4. V. Churavy, W. F. Godoy, C. Bauer, H. Ranocha, M. SchlottkeLakemper, L. Räss, J. Blaschke, M. Giordano, E. Schnetter, S. Omlin, *et al.*, “Bridging HPC communities through the Julia programming language,” *arXiv preprint arXiv:2211.02740*, 2022. Available at: <https://arxiv.org/abs/2211.02740>.
5. E. Carson and T. Mary, “Mixed-precision Computing: High Accuracy With Low Precision,” *SIAM News*, vol. 58, no. 9, November 2025.
6. CodeProject, “Floating Point and Integer Arithmetic Benchmark,” *CodeProject*, 2014. Available: <https://www.codeproject.com/articles/Floating-Point-and-Integer-Arithmetic-Benchmark>.
7. S. Desrochers, M. Dagenais, and H. Kettani, “Validation of DRAM RAPL Power Measurements,” in *Proceedings of the International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2016. doi: <https://doi.org/10.1145/2989081.2989088>.
8. G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed., Johns Hopkins University Press, 2013.
9. N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, SIAM, 2002.
10. S. Hunold and S. Steiner, “Benchmarking Julia’s communication performance: Is Julia HPC ready or full HPC?” in *Proceedings of the IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*, pp. 20–25, 2020.
11. Intel Community, “Integer Unit vs Floating Point Unit,” *Intel Community Forum*, 2018. Available: <https://community.intel.com/t5/Software-Tuning-Performance/Integer-Unit-vs-Floating-Point-Unit/m-p/1113147>.
12. IEEE Standard for Floating-Point Arithmetic, IEEE Std 754-2019 (Revision of IEEE 754-2008), IEEE Computer Society, 2019.
13. P. Kharya, “What is the TensorFloat-32 Precision Format?,” *NVIDIA Blog*, May 14, 2020. Available at: <https://blogs.nvidia.com/blog/tensorfloat-32-precision-format/>.
14. The Linux Kernel Documentation, “Power Capping Framework (Powercap),” *The Linux Kernel Documentation*, n.d. Available: <https://docs.kernel.org/power/powercap/powercap.html>
15. C. Lattner and V. Adve, “The LLVM Compiler Framework and Infrastructure Tutorial,” in *Languages and Compilers for High Performance Computing*, pp. 15–16, 2005. doi: https://doi.org/10.1007/11532378_2.
16. M. Lehmann, M. J. Krause, G. Amati, M. Sega, J. Harting, and S. Gekle, “On the accuracy and performance of the lattice Boltzmann method with 64-bit, 32-bit and novel 16-bit number formats,” *arXiv preprint arXiv:2112.08926*, 2022. Available at: <https://arxiv.org/abs/2112.08926>.
17. P. Micikevicius, S. Narang, J. Alben, *et al.*, “Mixed Precision Training,” *International Conference on Learning Representations (ICLR)*, 2018.
18. J. Muraña and S. Nesmachnow, “Empirical power consumption characterization and energy-aware scheduling in data centers,” *CLEI Electronic Journal*, vol. 24, no. 1, paper 3, pp. 1–28, Apr. 2021. Available: <https://doi.org/10.19153/cleiej.24.1.3>.
19. R. Pereira, M. Couto, F. Ribeiro, R. Rua, J. Cunha, J. P. Fernandes, and J. Saraiva, “Energy Efficiency across Programming Languages: How Do Energy, Time, and Memory Relate?,” in *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering (SLE)*, pp. 256–267, 2017. doi: <https://doi.org/10.1145/3136014.3136031>.
20. R. Sakamoto, M. Kondo, K. Fujita, T. Ichimura, and K. Nakajima, “The effectiveness of low-precision floating arithmetic on numerical codes: a case study on power consumption,” in *Proceedings of the International Conference on High Performance Computing in Asia-Pacific Region (HPC Asia)*, pp. 199–206, 2020.
21. Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed., SIAM, 2003.
22. SciComp StackExchange, “Integer Operations vs Floating Point Operations,” *Computational Science Stack Exchange*, 2019. Available: <https://scicomp.stackexchange.com/questions/30353/integer-operations-vs-floating-point-operations>.

23. Stack Overflow, "Floating Point vs Integer Calculations on Modern Hardware," *Stack Overflow*, 2010. Available: <https://stackoverflow.com/questions/2550281/floating-point-vs-integer-calculations-on-modern-hardware>.
24. J. M. R. Teichgräber, "Julia: A competitive high-level choice for performance portability in HPC?" in *MPCDF Seminar on (Performance) Portable Programming of HPC Applications*, 2022.
25. L. N. Trefethen and D. Bau III, *Numerical Linear Algebra*, SIAM, 1997.
26. N. van Kempen, H.-J. Kwon, D. T. Nguyen, and E. D. Berger, "It's Not Easy Being Green: On the Energy Efficiency of Programming Languages," *arXiv preprint arXiv:2410.05460*, 2024. Available at: <https://arxiv.org/abs/2410.05460>.
27. S. Wang and Q. Zhang, "BFloat16: The Secret to High Performance on Cloud TPUs," *Google Cloud Blog*, 2019.